

## — Web apps

*Chapter 8: PWA – introduction*

*L. Espeel*



## — Situering <sup>(1)</sup>

Tegenwoordig zijn er verschillende manieren om een app op een mobiel apparaat weer te geven:

| Type app                      | Technologie / Taal                                               | Engine                              | Ontwikkelingskost   | Toegang tot hardware                |
|-------------------------------|------------------------------------------------------------------|-------------------------------------|---------------------|-------------------------------------|
| Web App                       | HTML, CSS, JavaScript                                            | Browser                             | Laag                | Beperkt (via browser API's)         |
| PWA                           | HTML, CSS, JavaScript + Service Workers                          | Browser<br>WebView (bij publicatie) | Laag –<br>Gemiddeld | Gedeeltelijk (camera, notificaties) |
| Hybride App                   | HTML, CSS, JavaScript JavaScript (met Cordova, Ionic, Capacitor) | WebView + Native bridge             | Gemiddeld           | Via plug-ins                        |
| Native App (via React Native) | JavaScript/TypeScript + React                                    | React Native engine                 | Gemiddeld           | Volledige toegang via modules       |
| Echte Native App              | Java/Kotlin (Android)<br>Xcode/Swift (iOS)                       | Native SDK's                        | Hoog                | Volledige toegang                   |

## — Situering (2)

| Type app                         | Performantie                        | Publicatie in store              | Onderhoud & Updates                   | Offline ondersteuning      |
|----------------------------------|-------------------------------------|----------------------------------|---------------------------------------|----------------------------|
| Web App                          | Matig                               | Nee                              | Eenvoudig<br>(centrale website)       | Beperkt<br>(browsercacher) |
| PWA                              | Goed,<br>afhankelijk<br>van browser | Beperkt<br>(soms via Play Store) | Eenvoudig<br>(web push, updates)      | Ja<br>(offline cache)      |
| Hybride App                      | Goed                                | Ja                               | Matig<br>(afhankelijk van plugins)    | Ja                         |
| Native App<br>(via React Native) | Zeer goed                           | Ja                               | Matig<br>(per platform apart builden) | Ja                         |
| Echte Native App                 | Uitstekend                          | Ja                               | Moeilijker<br>(twee codebases)        | Ja                         |

3

## — PWA (1)



- Een Progressive Web App (PWA) is een webapplicatie gebouwd met HTML, CSS en JavaScript die zich gedraagt als een native app: het is installeerbaar op het startscherm, werkt (gedeeltelijk) offline en er zijn push-notificaties mogelijk.
- Een PWA is geen aparte technologie, maar een verzameling van best practices en API's die samen een app-achtig gedrag mogelijk maken.

4

## — PWA (2)

Belangrijke technische vereisten:

- PWA's moeten via HTTPS draaien om veiligheid en betrouwbaarheid te garanderen.
  - Via test op localhost is HTTPS niet vereist.
- Een valid web app manifest (bv. *manifest.json*) is nodig met minimaal de opties *name/short\_name*, *start\_url*, *display* (zoals standalone of fullscreen) en *icons*.
- Een *service worker* is geregistreerd om caching, offline functionaliteit en/of push-notificaties te ondersteunen.
- Goede iconen / splash screens / asset-generatie zijn nodig voor verschillende devices.

5

## — PWA (3)

Voordelen:

- Één codebase voor web + mobiel.
- Installeerbaar op het startscherm.
- Er kan offline gewerkt worden doordat er veel gecached wordt.
- Lagere ontwikkelkosten.
- Directe updates zonder store-goedkeuring.
  - Gebruikers hebben ook sneller de update van de app binnen.

Beperkingen:

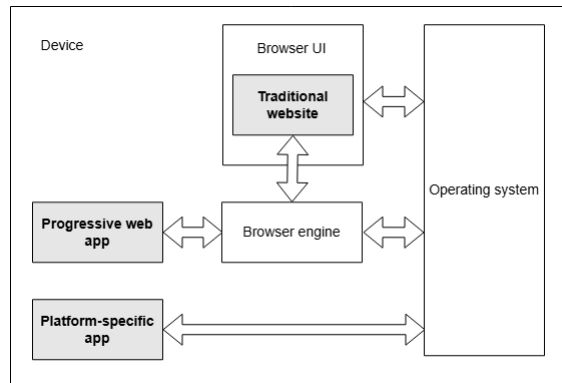
- Toegang tot sommige sensoren (zoals Bluetooth, NFC of achtergrondlocatie) is beperkt afhankelijk van het platform en de browser.

6

## — Overzicht

PWA's zien er meestal uit als platformspecifieke apps.

- Ze worden meestal weergegeven zonder de gebruikersinterface van de browser eromheen, maar het zijn qua technologie nog steeds websites.
- Dit betekent dat ze een browser-engine nodig hebben, zoals die in Chrome of Firefox, om ze te beheren en uit te voeren.



7

## — Componenten (1)

De belangrijkste componenten die je aantreft in PWA's zijn het volgende:

- **Service worker:**
  - Een JavaScript-bestand dat draait in de achtergrond van je browser en taken uitvoert zoals caching, pushmeldingen en offline ondersteuning.
  - Hoewel er voor een PWA geen service worker hoeft te worden geïnstalleerd, worden service workers vaak gebruikt met PWA's om op zijn minst een minimale offline ervaring te bieden.
  - Hoewel een service worker optioneel is, is deze verplicht om een PWA als "installable" te laten herkennen door de browser.

8

## — Componenten (2)

- Manifest:

- Een manifestbestand van een web-app bevat de minste informatie in JSON-formaat die de browser nodig heeft om de PWA te installeren, zoals de naam en het pictogram van de app.
- Het manifestbestand moet beschikbaar zijn via de root van je site (of met correcte *scope*), zodat browsers het kunnen detecteren.

```
{
  "name": "my-react-pwa-app",
  "short_name": "my-react-pwa-app",
  "description": "my-react-pwa-app",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#ffffff",
  "lang": "en",
  "scope": "/",
  "icons": [
    {
      "src": "pwa-64x64.png",
      "sizes": "64x64",
      "type": "image/png"
    },
    {
      "src": "pwa-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    }
  ]
}
```

9

## — Componenten (3)

- Workbox:

- Open-source JavaScript library van Google die het makkelijk maakt om PWA's te bouwen, vooral het beheer van caching en offline functionaliteit via service workers.
- Workbox automatiseert cachingstrategieën, zodat je minder snel fouten maakt bij handmatig cachebeheer.

```
workbox.routing.registerRoute(
  ({request}) => request.destination === 'image',
  new workbox.strategies.CacheFirst()
);
```

10

## — Vite



- Vite is een moderne ontwikkelservers en build-tool voor webprojecten, ontworpen om sneller en efficiënter te zijn dan traditionele bundlers. Het wordt veel gebruikt voor frameworks zoals React, Vue, ... en ook voor PWAs.
- De officiële PWA-plugin voor Vite is **vite-plugin-pwa**, gebaseerd op Workbox. Deze integreert naadloos met frameworks als React.
- Kenmerken:
  - Ontwikkelservers met instant hot module replacement (HMR).
  - Build-tool voor productie.
  - Optimalisaties: automatische code-splitting, tree-shaking van ongebruikte code, snellere builds dan oudere bundlers.

11

## — Project aanmaken (1)

- `npm create @vite-pwa/pwa@latest <name> -- --template react`

Opmerking: gebruik Node 18 of hoger voor compatibiliteit.  
Kies in de volgende wizard gerust de standaardopties, het is mogelijk om deze eventueel achteraf te veranderen in `vite.config.js`.

```
✓ PWA Name: ... my-react-pwa-app
✓ PWA Short Name: ... my-react-pwa-app
✓ PWA Description: ...
✓ Theme color: ... #ffffff
? Select a strategy: » - Use arrow-keys. Return to submit.
> generateSW
  injectManifest
```

### generateSW

Er wordt automatisch een service worker gegenereerd die cachingregels bevat voor statische assets (HTML, JS, CSS, afbeeldingen, ...) en offline-functionaliteit mogelijk maakt.

Gebruik generateSW als je de standaardconfiguratie van Workbox wilt gebruiken.

### injectManifest

Gebruik dit als je volledige controle wilt over de cachinglogica in je eigen `service-worker.js` bestand.

De Workbox tool injecteert automatisch een lijst van bestanden die in de cache moeten komen (het "precache manifest"). Hierdoor kan je eigen logica toevoegen zoals: achtergrond-sync, push notifications, ...

12

## — Project aanmaken (2)

```
? Select a behavior: » - Use arrow-keys. Return to submit.  
> Prompt for update  
Auto update
```

### Prompt for update

De gebruiker krijgt een melding wanneer er een nieuwe versie van de PWA beschikbaar is.

### Auto update

De service worker update automatisch op de achtergrond zodra een nieuwe versie beschikbaar is. De volgende keer dat de gebruiker de app opent, krijgt hij direct de nieuwe versie.

13

## — Project aanmaken (3)

```
? Enable periodic SW updates? » no / yes
```

Met periodic SW updates laat je de browser regelmatig op de achtergrond controleren op updates, zelfs als de gebruiker de app niet actief opent.

```
? Show offline ready prompt? » no / yes
```

**yes:** de app toont een melding of banner wanneer de service worker alles heeft gecachet wat nodig is om offline te werken. Bv. "App is nu offline beschikbaar".

**no:** geen melding. De app werkt offline, maar de gebruiker krijgt geen visuele bevestiging.

14

## — Project aanmaken (4)

? Generate PWA Assets Icons on the fly? » no / yes

**yes:** de tool genereert automatisch alle benodigde iconen en manifest-bestanden voor je PWA: app-icons in verschillende maten (voor iOS, Android, desktop, etc.), *manifest.json* met correcte verwijzingen naar de iconen, splash screens en andere PWA-gerelateerde assets.

Handig omdat je maar één bronicoon hoeft aan te leveren en de tool doet de rest.

**no:** je moet zelf alle icons en manifest-bestanden aanleveren. Dit geeft volledige controle over de assets, maar kost veel meer werk.

Dit kan nuttig zijn als je zeer specifieke branding wilt gebruiken of eigen optimalisaties.

15

## — vite.config.js (1)

In *vite.config.js* kan je opties toevoegen, aanpassen of verwijderen.

```
import { VitePWA } from 'vite-plugin-pwa';
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import mkcert from 'vite-plugin-mkcert'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react(), mkcert(), VitePWA({
    registerType: 'prompt',
    injectRegister: false,

    pwaAssets: {
      disabled: false,
      config: true,
    },

    manifest: {
      name: 'my-react-pwa-app',
      short_name: 'my-react-pwa-app',
      description: 'my-react-pwa-app',
      theme_color: '#ffffff'
    }
  })]
})
```

16



## — vite.config.js (2)

Zo voegden we de library *mkcert* toe (voer commando *npm install vite-plugin-mkcert -D* uit) die een certificaat/sleutel zal genereren om HTTPS op te zetten. Dit is nodig om de PWA-features te testen die dit vereisen (zoals service workers en installatieprompt).

```
    }],  
    server: {  
      https: true, // als je mkcert gebruikt  
      host: '0.0.0.0', // om het even welk ip-adres connecteert met deze server  
    },  
  },  
})
```

Het server-deel bevat de optie om HTTPS te activeren en connectie toe te laten van om het even welk ip-adres.

Meer info over dit bestand kan je best raadplegen via <https://vite.dev/config>.

17

## — Builden en uitvoeren (1)

Als de wizard ten einde is, voer dan deze commando's uit:

- *cd <name>*
- *npm install*

Indien je snel wenst uit te **testen** in de browser:

- *npm run dev*
  - Open een browser en surf naar <http://localhost:5173>.

18

## — Builden en uitvoeren (2)

Als je de PWA wilt builden voor productie:

- `npm run build`
- `npm run preview`
  - Is een simulatie van de productieomgeving, handig voor het testen van cachinggedrag.
  - Open browser naar <http://localhost:4173>.
  - Hier is het mogelijk om de Manifest en Service workers te bekijken (te vinden bij dev tools via F12).

19

hogeschool  
**vives**

## — Builden en uitvoeren (3)

The screenshot shows the Chrome DevTools Application tab with three main panels:

- Manifest:** Displays the `manifest.webmanifest` file. It includes "Errors and warnings" (Richer PWA Install UI won't be available on desktop/mobile) and "Identity" (Name: my-react-pwa-app, Short name: my-react-pwa-app, Description: my-react-pwa-app).
- Storage:** Lists various storage areas including Local storage, Session storage, Extension storage, IndexedDB, Cookies, Private state tokens, Interest groups, Shared storage, Cache storage, Storage buckets, Background services, Background fetch, Background sync, Bounce tracking, Notifications, and Payment handler.
- Service workers:** Shows the service worker for `http://localhost:4173/`. It indicates that the worker is activated and running. It also shows a "Push" message and a "Periodic sync" event.

| Version | Update Activity | Timeline |
|---------|-----------------|----------|
| #2515   | Install         |          |
| #2515   | Wait            |          |
| #2515   | Activate        |          |

20

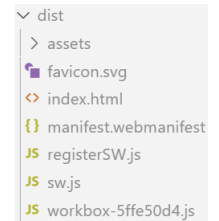
## — Gegeneerde bestanden (1)

Na het bouwen naar productie werd het één en ander aangemaakt onder de map *dist*.

- **manifest.webmanifest**: het manifest-gedeelte uit *vite.config.js* is hierin opgenomen.
- **registerSW.js**: klein JavaScript-bestand dat de service worker registreert in de browser.

```
if ('serviceWorker' in navigator) {  
  window.addEventListener('load', () => {  
    navigator.serviceWorker.register('/sw.js', { scope: '/' })  
  })  
}
```

- **sw.js**: de daadwerkelijke service worker van de PWA.
- **workbox.js**: de runtime library van Workbox, biedt helper functies om cachingstrategieën en precaching te beheren.



21

## — Gegeneerde bestanden (2)

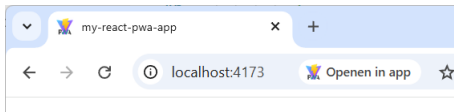
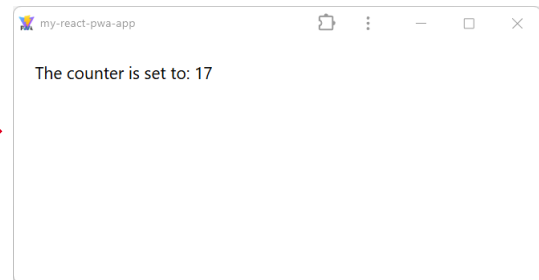
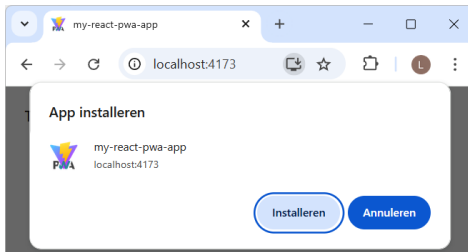
- **index.html**: het minimale voor een PWA is opgenomen met o.a. viewport, verwijzingen naar manifest, icoontjes en registerSW.

```
<!doctype html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <title>my-react-pwa-app</title>  
    <script type="module" crossorigin src="/assets/index-BTjIZmBC.js"></script>  
    <link rel="stylesheet" crossorigin href="/assets/index-BvKRaj3g.css">  
    <link rel="manifest" href="/manifest.webmanifest">  
    <script id="vite-plugin-pwa:register-sw" src="/registerSW.js"></script></head>  
    <body>  
      <div id="root"></div>  
    </body>  
</html>
```

22

## Uittesten (1)

Mogelijkheid om de app te installeren en later te heropenen:

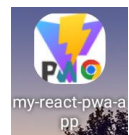
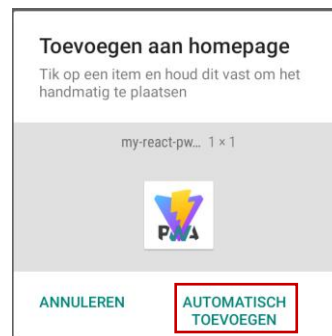
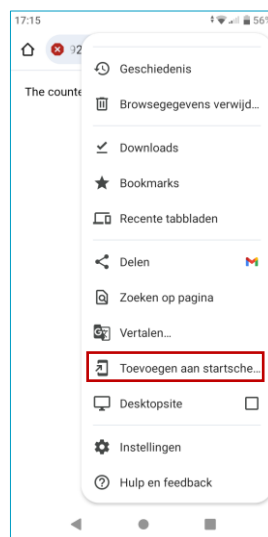
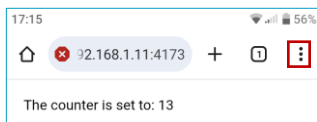


23

hogeschool  
**vives**

## Uittesten (2)

Mobile device:



24

hogeschool  
**vives**

## — Uittesten <sup>(3)</sup>

- Eenmaal geïnstalleerd, gedraagt de PWA zich net als andere geïnstalleerde applicaties: door op het applicatiepictogram te klikken, wordt de PWA geopend, zelfs als de gebruiker offline is.
- PWA's die door een browser worden geïnstalleerd, blijven specifiek voor deze browser. Dit betekent dat de browser die is gebruikt om een PWA te installeren, de browser is die is gebruikt om die PWA uit te voeren. Het betekent ook dat je dezelfde PWA vanuit een andere browser kunt installeren en dat de twee apps zich als twee verschillende instanties gedragen en geen gegevens delen.
- Op de meeste mobiele besturingssystemen wordt het verwijderen van een PWA op dezelfde manier bereikt als het verwijderen van andere applicaties.

25

## — Publicatie <sup>(1)</sup>

PWA's hoeven in principe niet gepubliceerd te worden in een store om ze te installeren, doch is het een goed idee om dit te doen:

- Bereik: veel gebruikers zoeken apps via stores, dus listing kan zichtbaarheid vergroten.
- Vertrouwen: gebruikers vinden het vaak betrouwbaarder om via een bekende store te installeren.
- Installatiegemak: store-installatie, iconen op startscherm, automatische updates.
- Distributie-optie: je kunt zowel webversie (via URL) als store-versie aanbieden.
- Nadeel: voor store-publicatie moet je voldoen aan store-richtlijnen en soms een wrapper of extra tool gebruiken (meer werk).

26

## — Publicatie (2)

Om een PWA te kunnen publiceren in een app store wordt deze "verpakt" als een native container die de webapp binnen een WebView uitvoert. De PWA blijft dus draaien in een browser-engine.

Volg de stappen om dit mogelijk te maken:

- [Google Play](#)
- [Microsoft Store](#)
- [Meta Quest Store](#)

Opmerking: Apple ondersteunt PWA's in Safari, maar accepteert ze officieel niet direct in de App Store. Je moet dan een wrapper gebruiken, zoals Capacitor (van Ionic), Cordova en PWABuilder.

27

## — Referenties

Doorneem deze website voor meer info over Vite React PWA:

- <https://vite-pwa-org.netlify.app/guide>

Algemene info over PWA:

- <https://web.dev/pwa>
- [https://developer.mozilla.org/en-US/docs/Web/Progressive\\_web\\_apps](https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps)

28